4. 자바스크립트 객체

4-1. 객체란 ?

- **객체(Object) 란**
  - 프로그램에서 인식할 수 있는 모든 대상
  - 데이터를 저장하고 처리하는 기본 단위
- **자바스크립트 객체**
  - 자바스크립트 안에 미리 객체로 정의해 놓은 것
  - **문서 객체 모델(DOM)** : 문서 뿐만 아니라 웹 문서 안에 포함된 이미지·링크·텍스트 필드 등을 모두 별도의 객체로 관리
  - **브라우저 관련 객체** : 웹 브라우저 정보를 객체로 관리
  - **내장 객체** : 웹 프로그래밍에서 자주 사용하는 요소를 객체로 정의해 놓음
- **사용자 정의 객체**
  - 사용자자 만든 객체

- **객체와 인스턴스**
  - 객체는 객체 자체가 아니라 인스턴스 형태로 만들어서 사용
  - 인스턴스 : 객체를 틀처럼 사용해서 같은 모양으로 찍어낸 것

```
// 현재 날짜와 시간 정보 표시하기
<script>
  let now = new Date(); // 인스턴스 객체 만들고 변수에 할당하기
  document.write("현재 시각은 " + now) ; // 현재 날짜와 시간 표시하기
</script>
```

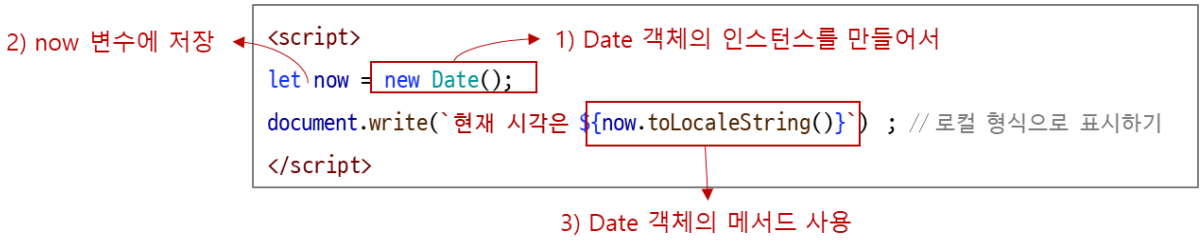
- **프로퍼티(property)와 메서드(method)**
  - 프로퍼티 : 객체의 특징이나 속성

- 메서드 : 객체에서 할 수 있는 동작
- 인스턴스는 객체의 프로퍼티와 메서드를 그대로 물려받음
- 프로퍼티와 메서드를 표시하려면 인스턴스명 뒤에 마침표(.)를 붙이고 프로퍼티나 메서드 이름 작성



프로퍼티	메서드
제조사	시동 걸기
모델명	움직이기
색상	멈추기
배기량	주차하기

그림 16-2 자동차의 프로퍼티와 메서드



4-2. 내장객체

1. Array 객체

```
// 1. 초기값이 없는 경우
let arr1 = new Array(); // 배열의 크기를 지정하지 않음
let arr2 = new Array(4); // 배열의 크기를 지정함

// 2. 초기 값이 있는 경우
<script>
  let numbers = ["one", "two", "three", "four"];    // 배열 선언 및 초기화
```

```
  for(let i = 0; i < numbers.length; i++) {          // 배열
의 각 요소에 접근하기
    document.write(`<p>${numbers[i]}</p>`);
  }
</script>
```

종류	설명
concat	기존 배열에 요소를 추가해 새로운 배열을 만듭니다.
every	배열의 모든 요소가 주어진 함수에 대해 참이면 true를 반환하고 그렇지 않으면 false를 반환합니다.
filter	배열 요소 중에서 주어진 필터링 함수에 대해 true인 요소만 골라 새로운 배열을 만듭니다.
forEach	배열의 모든 요소에 대해 주어진 함수를 실행합니다.
indexOf	주어진 값과 일치하는 값이 있는 배열 요소의 첫 인덱스를 찾습니다.
join	배열 요소를 문자열로 합칩니다. 이때 구분자를 지정할 수 있습니다.
push	배열의 맨 끝에 새로운 요소를 추가한 후 새로운 length를 반환합니다.
unshift	배열의 시작 부분에 새로운 요소를 추가합니다.
pop	배열의 마지막 요소를 꺼내 그 값을 결과로 반환합니다.
shift	배열에서 첫 번째 요소를 꺼내 그 값을 결과로 반환합니다.
splice	배열에 요소를 추가하거나 삭제합니다.
slice	배열에서 특정한 부분만 잘라 냅니다.
reverse	배열의 배치 순서를 역순으로 바꿉니다.
sort	배열 요소를 지정한 조건에 따라 정렬합니다.
toString	배열에서 지정한 부분을 문자열로 반환합니다. 이때 각 요소는 쉼표(,)로 구분합니다.

- **배열끼리 합치는 concat( ) 메서드**

```
<script>
  let nums = [1, 2, 3];
  let chars = ['a', 'b', 'c', 'd’];

  let numsChars = nums.concat(chars);
  let charsNums = chars.concat(nums);
  document.write(`nums에 chars 합치면 : ${numsChars}, <br
> chars에 nums 합치면  ${charsNums}`);
  document.write(`<hr>`);
</script>
```

- **배열 요소끼리 합치는 join( ) 메서드**

```
<script>
  let nums = [1, 2, 3];
  let chars = ['a', 'b', 'c', 'd’];
   ......
  let string1 = nums.join();
  document.write(`구분자 없이 : ${string1}<br>`);
  let string2 = chars.join(‘/’);
  document.write(`/ 구분자 지정 : ${string2}`);
  document.write(`<hr>`);
</script>
```

- **새로운 요소를 추가하는 push( ), unshift( ) 메서드**

```
<script>
  let nums = [1, 2, 3];
  let chars = ['a', 'b', 'c', 'd’];
     ......

  let ret1 = nums.push(4, 5);   // 배열 끝에 추가
  document.write(`length : ${ret1} | 배열 : ${nums}<br>
`);
  let ret2 = nums.unshift(0);   // 배열 앞에 추가
  document.write(`length : ${ret2}, | 배열 : ${nums}`);
  document.write(`<hr>`);
</script>
```

- **배열에서 요소를 꺼내는 pop( ), shift( ) 메서드**

```
<script>
  let nums = [1, 2, 3];
  let chars = ['a', 'b', 'c', 'd’];
      ......
  let popped1 = chars.pop();   // 마지막 요소 꺼냄
  document.write(`꺼낸 요소 : ${popped1}, | 배열 : ${char
s}<br>`);
```

```
  let popped2 = chars.shift();   // 첫 번째 요소 꺼냄
  document.write(`꺼낸 요소 : ${popped2}, | 배열 : ${char
s}`);
  document.write(`<hr>`);
</script>
```

- **중간에 요소를 추가하거나 삭제하는 splice( ) 메서드**

```
<script>
  let study = ['html', 'css', 'javascript', 'jquery', 'r
eact', 'nodejs'];

  // 인수가 1개일 경우
  let js = study.splice(2);
  document.write(`반환된 배열 : ${js}<br>` );
  document.write(`변경된 배열 : ${study}`);
  document.write(`<hr>`);

  // 인수가 2개일 경우
  let jquery = js.splice(1,1);
  document.write(`반환된 배열 : ${jquery}<br>`);
  document.write(`변경된 배열 : ${js}`);
  document.write(`<hr>`);

  // 인수가 3개 이상일 경우
  let modernJs = js.splice(1, 0, 'typescript');
  document.write(`반환된 배열 : ${modernJs}<br>`);
  document.write(`변경된 배열 : ${js}`);
</script>
```

- **기존 배열을 바꾸지 않으면서 추출하는 slice( ) 메서드**

```
<script>
  let colors = ["red", "green", "blue", "white", "blac
k"];

  document.write(colors.slice(2)); //인덱스 2부터 끝까지
```

```
  document.write("<hr>");
  document.write(colors.slice(2, 4)); // 인덱스 2, 3
</script>
```

💡 slice() 메서드는 기존 배열에 영향을 주지 않지만,
splice() 메서드는 요소를 추가·삭제하면 기존 배열 자체가 수정됨

→ 기존 배열에서 꺼낸 요소로 새로운 배열을 만들어 사용하려면 slice() 메서드를 사용하고,
기존 배열의 일부 요소만 삭제하려면 splice() 메서드를 선택하는 것이 좋다

2. Date객체

```
new Date('2024-02-25');
new Date('2024-02-25T18:00:00)'

new Date("02/25/2024")
new Date("Mon Jan 20 2024 15:00:41 GMT+0900 (대한민국 표준시)")
```

구분		설명
날짜·시간 정보 가져오기	getFullYear()	연도를 4자리 숫자로 표시합니다.
	getMonth()	0~11 사이의 숫자로 월을 표시합니다. 0부터 1월이 시작되고 11은 12월입니다.
	getDate()	1~31 사이의 숫자로 일을 표시합니다.
	getDay()	0~6 사이의 숫자로 요일을 표시합니다. 0부터 일요일이 시작되고 6은 토요일입니다.
	getTime()	1970년 1월 1일 자정 이후의 시간을 밀리 초(1/1000초)로 표시합니다.
	getHours()	0~23 사이의 숫자로 시를 표시합니다.
	getMinutes()	0~59 사이의 숫자로 분을 표시합니다.
	getSeconds()	0~59 사이의 숫자로 초를 표시합니다.
	getMilliseconds()	0~999 사이의 숫자로 밀리초를 표시합니다.
날짜·시간 설정하기	setFullYear()	연도를 4자리 숫자로 설정합니다.
	setMonth()	0~11 사이의 숫자로 월을 설정합니다. 0부터 1월이 시작되고 11은 12월입니다.
	setDate()	1~31 사이의 숫자로 일을 설정합니다.
	setTime()	1970년 1월 1일 자정 이후의 시간을 밀리초로 설정합니다.
	setHours()	0~23 사이의 숫자로 시를 설정합니다.
	setMinutes()	0~59 사이의 숫자로 분을 설정합니다.
	setSeconds()	0~59 사이의 숫자로 초를 설정합니다.
	setMilliseconds()	0~999 사이의 숫자로 밀리초를 설정합니다.
날짜·시간 형식 바꾸기	toLocaleString()	현재 날짜와 시간을 현지 시간(local time)으로 표시합니다.
	toString()	Data 객체 타입을 문자열로 표시합니다.

▼ 예제 코드

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, ini
tial-scale=1.0">
  <title>독서 챌린지</title>
  <style>
```

```
    body {
      display: flex;
      flex-direction: column;
      justify-content: center;
      align-items: center;
      height: 100vh;
    }
    .container{
      width:300px;
      height:300px;
      border-radius:50%;
      border:1px solid #222;
      background-color:#3bf;
      text-align: center;
    }
    h1 {
      margin-top:70px;
    }
    .accent {
      font-size:1.8em;
      font-weight:bold;
      color:red;
    }
  </style>
</head>
<body>
  <div class="container">
    <h1>책 읽기</h1>
    <p><span class="accent" id="result"></span>일 연속으로
<br> 책 읽기를 달성했군요.</p>
    <p>축하합니다!</p>
  </div>

  <script>
    // Do it! Date 객체 연습하기

  </script>
```

```
</body>
</html>
```

3. Math 객체

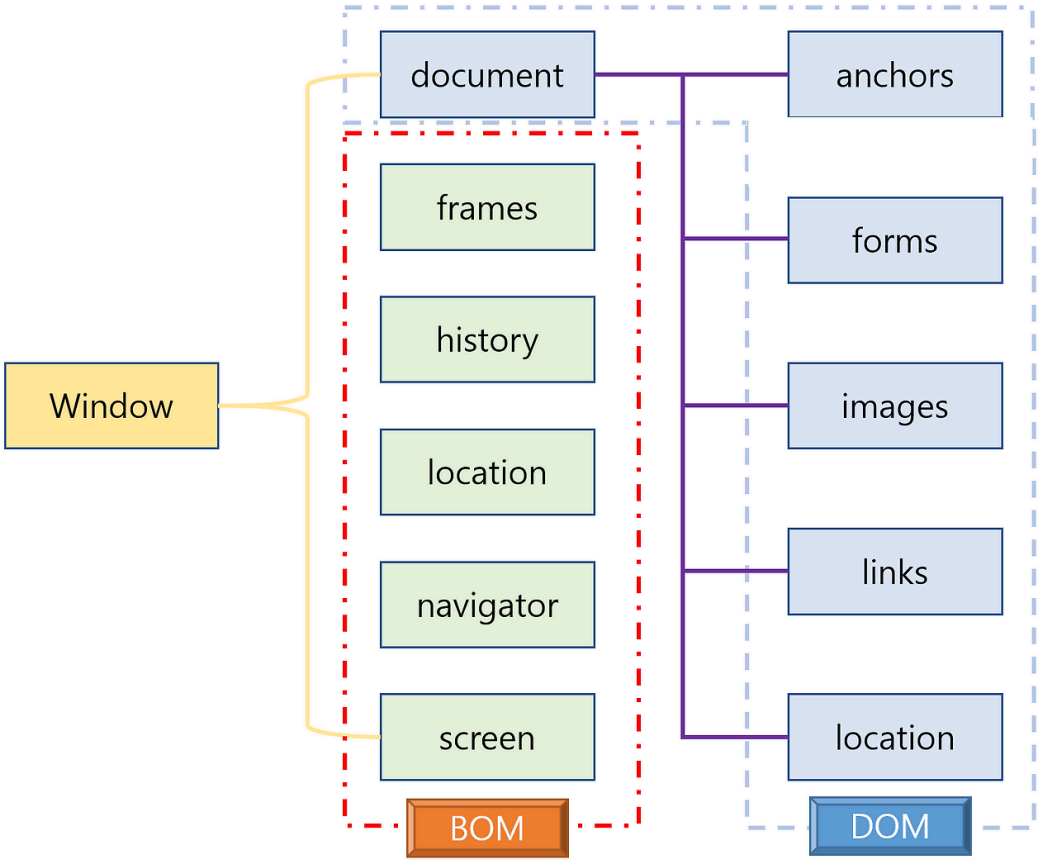
- 특징
  - 수학 계산과 관련된 메서드가 많이 포함되어 있지만 수학식에서만 사용하는 것은 아님.
  - 무작위 수가 필요하거나 반올림이 필요한 프로그램 등에서도 Math 객체의 메서드 사용함
  - Math 객체는 인스턴스를 만들지 않고 프로퍼티와 메서드 사용

종류	설명
E	오일러 상수
PI	원주율(π) (약 3.1415926535897930의 값)
SQRT2	√2(약 1.41421356237309510의 값)
SQRT1_2	1/√2(약 0.70710678118654760의 값)
LN2	$\log_e 2$(약 0.69314718055994530의 값)
LN10	$\log_e 10$(약 2.30258509299404060의 값)
LOG2E	$\log_2 e$(약 1.44269504088896340의 값)
LOG10E	$\log_{10} e$(약 0.43429448190325180의 값)

종류	설명
abs()	절댓값을 반환합니다.
acos()	아크 코사인(arc cosine)값을 반환합니다.
asin()	아크 사인(arc sine)값을 반환합니다.
atan()	아크 탄젠트(arc tangent)값을 반환합니다.
atan2()	아크 탄젠트(arc tangent)값을 반환합니다.
ceil()	매개변수의 소수점 이하 부분을 올립니다.
cos()	코사인(cosine)값을 반환합니다.
exp()	지수 함수를 나타냅니다.
floor()	매개변수의 소수점 이하 부분을 버립니다.
log()	매개변수에 대한 로그(log)값을 반환합니다.
max()	매개변수 중 최댓값을 반환합니다.
min()	매개변수 중 최솟값을 반환합니다.
pow()	매개변수의 지숫값을 반환합니다.
random()	0과 1 사이의 무작위 수를 반환합니다.
round()	매개변수의 소수점 이하 부분을 반올림합니다.
sin()	사인(sine)값을 반환합니다.
sqrt()	매개변수에 대한 제곱근을 반환합니다.
tan()	탄젠트(tangent)값을 반환합니다.

4-3 브라우저와 관련된 객체

- 브라우저 관련 객체의 계층 구조



종류	설명
window	브라우저 창이 열릴 때마다 하나씩 만들어집니다. 브라우저 창 안의 요소 중에서 최상위에 있습니다.
document	웹 문서마다 하나씩 있으며 <body> 태그를 만나면 만들어집니다. HTML 문서의 정보가 담겨 있습니다.
navigator	현재 사용하는 브라우저의 정보가 들어 있습니다.
history	현재 창에서 사용자의 방문 기록을 저장합니다.
location	현재 페이지의 URL 정보가 담겨 있습니다.
screen	현재 사용하는 화면 정보를 다룹니다.

- **window 객체의 프로퍼티**

종류	설명
document	브라우저 창에 표시된 웹 문서에 접근할 수 있습니다.
frameElement	현재 창이 다른 요소 안에 포함되어 있을 경우 그 요소를 반환하고, 반대로 포함되어 있지 않으면 null을 반환합니다.
innerHeight	내용 영역의 높이를 나타냅니다.
innerWidth	내용 영역의 너비를 나타냅니다.
localStorage	웹 브라우저에서 데이터를 저장하는 로컬 스토리지를 반환합니다.
location	window 객체의 위치 또는 현재 URL을 나타냅니다.
name	브라우저 창의 이름을 가져오거나 수정합니다.
outerHeight	브라우저 창의 바깥 높이를 나타냅니다.
outerWidth	브라우저 창의 바깥 너비를 나타냅니다.
pageXOffset	스크롤했을 때 수평으로 이동하는 픽셀 수로 scrollX와 같습니다.
pageYOffset	스크롤했을 때 수직으로 이동하는 픽셀 수로 scrollY와 같습니다.
parent	현재 창이나 서브 프레임의 부모입니다.
screenX	브라우저 창의 왼쪽 테두리가 모니터 왼쪽 테두리에서 떨어져 있는 거리를 나타냅니다.
screenY	브라우저 창의 위쪽 테두리가 모니터 위쪽 테두리에서 떨어져 있는 거리를 나타냅니다.
scrollX	스크롤했을 때 수평으로 이동하는 픽셀 수를 나타냅니다.
scrollY	스크롤했을 때 수직으로 이동하는 픽셀 수를 나타냅니다.
sessionStorage	웹 브라우저에서 데이터를 저장하는 세션 스토리지를 반환합니다.

- **window 객체의 메서드**

종류	설명
alert()	알림 창을 표시합니다.
blur()	현재 창에서 포커스를 제거합니다.
close()	현재 창을 닫습니다.
confirm()	[확인], [취소] 버튼이 있는 확인 창을 표시합니다.
focus()	현재 창에 포커스를 부여합니다.
moveBy()	현재 창을 지정한 크기만큼 이동합니다.
moveTo()	현재 창을 지정한 좌표로 이동합니다.
open()	새로운 창을 엽니다.
postMessage()	메시지를 다른 창으로 전달합니다.
print()	현재 문서를 인쇄합니다.
prompt()	프롬프트 창에 입력한 텍스트를 반환합니다.
resizeBy()	지정한 크기만큼 현재 창의 크기를 조절합니다.
resizeTo()	동적으로 브라우저 창의 크기를 조절합니다.
scroll()	문서에서 특정 위치로 스크롤합니다.
scrollBy()	지정한 크기만큼씩 스크롤합니다.
scrollTo()	지정한 위치까지 스크롤합니다.
sizeToContent()	내용에 맞게 창의 크기를 맞춥니다.
stop()	로딩을 중지합니다.

- **navigator 객체**
  - 사용하는 브라우저가 많아지고, 웹 애플리케이션이 등장하면서 navigator 객체에 여러 프로퍼티가 등장하고 있음.
  - 일부 브라우저에서만 지원하는 프로퍼티도 있음

종류	설명
battery	배터리 충전 상태를 알려 줍니다.
cookieEnabled	쿠키 정보를 무시하면 false, 허용하면 true를 반환합니다.
geolocation	모바일 기기를 이용한 위치 정보를 나타냅니다.
language	브라우저 UI의 언어 정보를 나타냅니다.
oscpu	현재 운영체제 정보를 나타냅니다.
userAgent	현재 브라우저 정보를 담고 있는 사용자 에이전트 문자열입니다.

- **history 객체**
  - 방문한 사이트 주소가 배열 형태로 저장됨

구분		설명
프로퍼티	length	현재 브라우저 창의 history 목록에 있는 항목의 개수, 즉 방문한 사이트 개수가 저장됩니다.
메서드	back()	history 목록에서 이전 페이지를 현재 화면으로 불러옵니다
	forward()	history 목록에서 다음 페이지를 현재 화면으로 불러옵니다
	go()	history 목록에서 현재 페이지를 기준으로 상대적인 위치에 있는 페이지를 현재 화면으로 불러옵니다. 예를 들어 history.go(1)은 다음 페이지를 가져오고, history.go(-1)은 이전 페이지를 불러옵니다.

- **location 객체**
  - 현재 문서의 URL 주소 정보가 담겨 있음
  - 이 정보를 편집해서 브라우저 창에 열 사이트/문서 지정

구분		설명
프로퍼티	hash	URL 중에서 #로 시작하는 해시 부분의 정보를 담고 있습니다.
	host	URL의 호스트 이름과 포트 번호를 담고 있습니다.
	hostname	URL의 호스트 이름이 저장됩니다.
	href	전체 URL입니다. 이 값을 변경하면 해당 주소로 이동할 수 있습니다.
	pathname	URL 경로가 저장됩니다.
	port	URL의 포트 번호를 담고 있습니다.
	protocol	URL의 프로토콜을 저장합니다.
	password	도메인 이름 앞에 username과 password를 함께 입력해서 접속하는 사이트의 URL일 경우에 password 정보를 저장합니다.
	search	URL 중에서 ?로 시작하는 검색 내용을 저장합니다.
	username	도메인 이름 앞에 username을 함께 입력해서 접속하는 사이트의 URL일 경우에 username 정보를 저장합니다.
메서드	assign()	현재 문서에 새 문서 주소를 할당해서 새 문서를 가져옵니다.
	reload()	현재 문서를 다시 불러옵니다.
	replace()	현재 문서의 URL을 지우고 다른 URL의 문서로 교체합니다.
	toString()	현재 문서의 URL을 문자열로 반환합니다.

- 예제 코드 (location 객체)

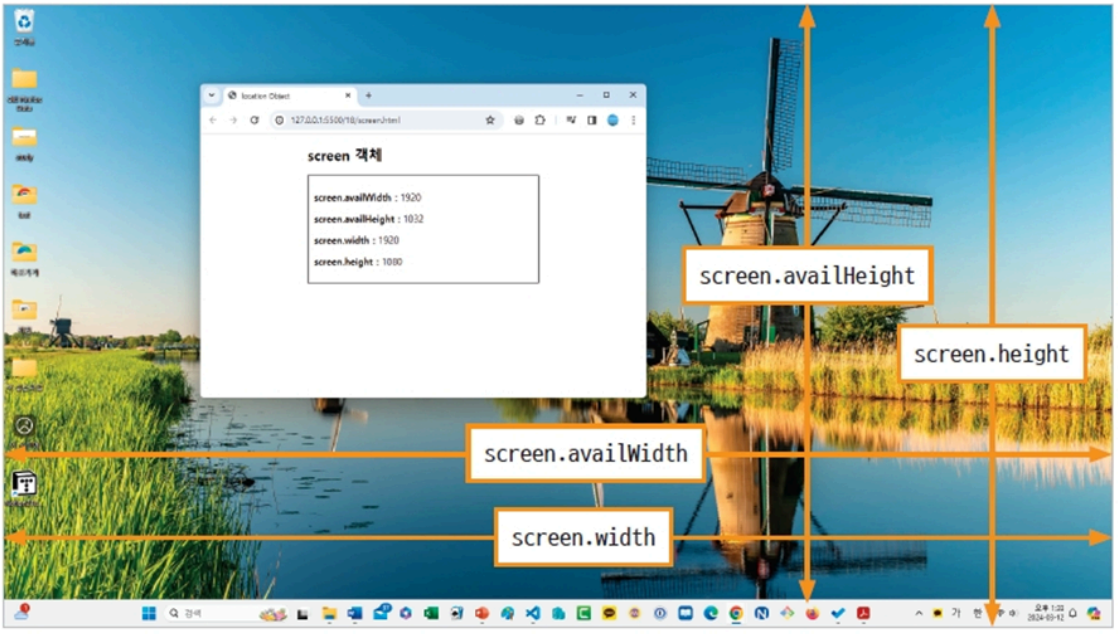
```
<!DOCTYPE html>
<html lang="ko">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, i
nitial-scale=1.0">
    <title>location Object</title>
    <style>
        .container {
            width:500px;
            margin:10px auto;
        }
        #display {
            margin-top:10px;
            padding:10px;
            border:1px solid #222;
```

```
            box-shadow: 1px 0 1px #ccc;
        }
        p {
            font-size:1em;
        }
        button {
            margin-top:20px;
            text-align:center;
        }
    </style>
</head>
<body>
    <!-- Do it! Location 객체 연습하기 - 현재 사이트에서 뒤로 가기 막기 -->
    <div class="container">
        <h2>location 객체 </h2>
        <div id="display">
            <script>

            </script>
        </div>
        <button>이지스퍼블리싱 홈페이지로 이동하기</button>
</div>
</body>
</html>
```

- **screen 객체**
  - 사용자의 화면 크기, 정보

구분		설명
프로퍼티	availHeight	UI 영역(예를 들어 윈도우의 작업 표시줄이나 Mac의 독)을 제외한 영역의 높이를 나타냅니다.
	availWidth	UI 영역을 제외한 내용 표시 영역의 너비를 나타냅니다.
	colorDepth	화면에서 픽셀을 렌더링할 때 사용하는 색상 수를 나타냅니다.
	height	UI 영역을 포함한 화면의 높이를 나타냅니다.
	orientation	화면의 현재 방향을 나타냅니다.
	pixelDepth	화면에서 픽셀을 렌더링할 때 사용하는 비트 수를 나타냅니다.
	width	UI 영역을 포함한 화면의 너비를 나타냅니다.
메서드	lockOrientation()	화면 방향을 잠급니다.
	unlockOrientation()	화면 방향 잠금을 해제합니다.

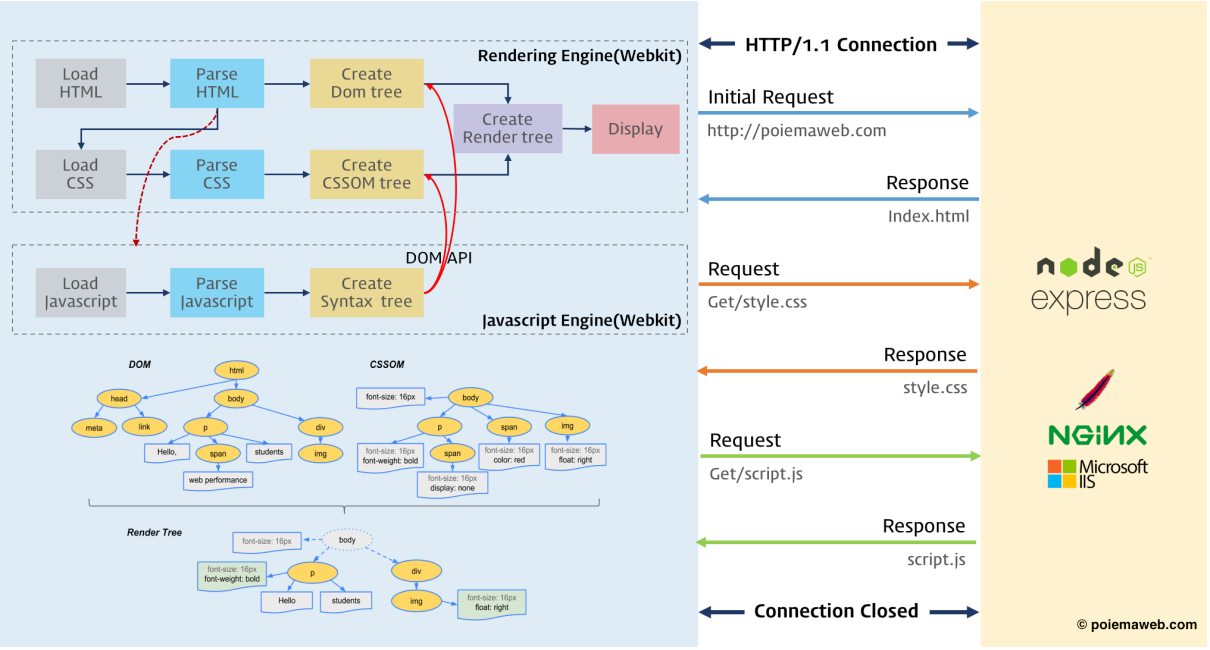


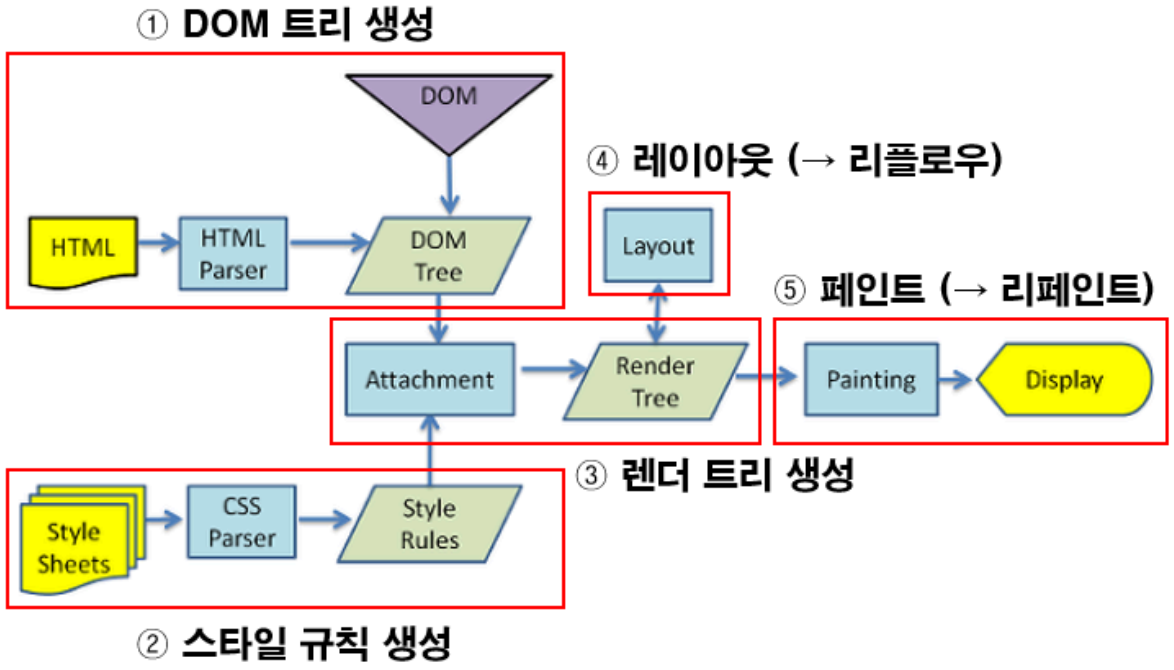
notice.html

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>공지사항</title>
  <style>
    .container {
      border : 2px double skyblue;
      border-radius:10px;
```

```
      padding:10px;
    }
    ul {
      margin-left:15px;
      list-style-type:none;
    }
    ul li {
      margin : 10px 5px;
    }
    button{
      position:absolute;
      bottom:20px;
      right:20px;
    }
  </style>
</head>
<body>
  <!-- Do it! '닫기' 버튼에 close() 메서드 연결하기 -->
  <div class="container">
    <h1>공지사항</h1>
    <ul>
      <li>항목 1</li>
      <li>항목 2</li>
      <li>항목 3</li>
      <li>항목 4</li>
      <li>항목 5</li>
    </ul>
    <button onclick="javascript:window.close();">닫기</butt
on>
  </div>
</body>
</html>
```

브라우저의 렌더링 과정


Rendering Engine(Webkit)
Load HTML
Parse HTML
Create Dom tree
Create Render tree
Display
Load CSS
Parse CSS
Create CSSOM tree
DOM API
Load Javascript
Parse Javascript
Create Syntax tree
Javascript Engine(Webkit)
DOM
CSSOM
Render Tree
HTTP/1.1 Connection
Initial Request
http://poiemaweb.com
Response
Index.html
Request
Get/style.css
Response
style.css
Request
Get/script.js
Response
script.js
Connection Closed
express
NGINX
Microsoft IIS
© poiemaweb.com
Client
Server


① DOM 트리 생성
DOM
HTML
HTML Parser
DOM Tree
④ 레이아웃 (→ 리플로우)
Layout
⑤ 페인트 (→ 리페인트)
Attachment
Render Tree
Painting
Display
③ 렌더 트리 생성
Style Sheets
CSS Parser
Style Rules
② 스타일 규칙 생성